

Guia geral e aprofundado: JWT + Roles em FastAPI para projetos reais

Objetivo do guia

Este guia explica, de forma conceitual e prática, como projetar e implementar autenticação com JWT e autorização baseada em papéis, também chamada de RBAC, em uma API FastAPI usada em um projeto real.

A intenção não é apenas mostrar código. O objetivo é construir um modelo mental sólido: o que é identidade, por que senhas precisam de hash, o que um token JWT realmente garante, o que ele não garante, onde entram roles, quais ameaças precisam ser consideradas e como transformar tudo isso em uma arquitetura limpa dentro de uma aplicação FastAPI.

Ao final, você deve ser capaz de:

- Explicar a diferença entre autenticação e autorização.
- Entender por que salvar senha em texto puro é uma falha grave.
- Usar bcrypt corretamente para armazenar senhas.
- Entender a estrutura de um JWT e suas claims principais.
- Projetar access token e refresh token com responsabilidades diferentes.
- Implementar login, refresh, usuário atual e proteção de rotas.
- Aplicar autorização por roles em endpoints FastAPI.
- Identificar limitações dessa abordagem e melhorias para produção.

Referências técnicas

Este guia se apoia nas seguintes referências:

- FastAPI Security: OAuth2 with Password and Bearer with JWT tokens: <https://fastapi.tiangolo.com/tutorial/security/oauth2-jwt/>
- FastAPI Dependencies: <https://fastapi.tiangolo.com/tutorial/dependencies/>
- FastAPI Security Scopes: <https://fastapi.tiangolo.com/advanced/security/oauth2-scopes/>
- RFC 7519 - JSON Web Token: <https://www.rfc-editor.org/rfc/rfc7519>
- OWASP JSON Web Token Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html
- OWASP Authorization Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html
- bcrypt no PyPI: <https://pypi.org/project/bcrypt/>

Parte 1: fundamentos

1. Autenticação não é autorização

A confusão mais comum em sistemas com login é tratar autenticação e autorização como se fossem a mesma coisa. Elas se relacionam, mas respondem perguntas diferentes.

Autenticação responde:

> Quem é você?

Autorização responde:

> O que você pode fazer?

Um exemplo simples:

- Lucas faz login com e-mail e senha.
- A API confirma que Lucas é realmente Lucas.

- Isso $\emptyset$ autenticação.

Depois:

- Lucas tenta deletar um usuário.
- A API verifica se Lucas tem papel 'admin'.
- Isso $\emptyset$ autorização.

Uma pessoa pode estar autenticada e ainda assim não estar autorizada. Esse $\emptyset$ um ponto essencial. Login bem-sucedido não significa acesso irrestrito ao sistema.

2. O ciclo de identidade em uma API

Em uma API HTTP, cada requisição $\emptyset$ independente. O servidor não lembra automaticamente quem chamou a rota anterior. Por isso, o cliente precisa enviar alguma credencial a cada chamada protegida.

Com JWT, o fluxo típico $\emptyset$:

1. Cliente envia e-mail e senha para '/auth/login'.
2. API valida as credenciais.
3. API emite um token assinado.
4. Cliente guarda esse token.
5. Cliente envia o token em rotas protegidas.
6. API valida o token em cada requisição.
7. API identifica o usuário.
8. API aplica regras de autorização.

Header padrão:

Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6...

O termo 'Bearer' significa, de forma prática, quem carrega este token pode usá-lo. Por isso, roubar um bearer token $\emptyset$ perigoso. Ele deve ser tratado como credencial sensível.

3. Senhas: o erro de salvar texto puro

Salvar senha em texto puro $\emptyset$ uma das falhas mais graves que uma API pode cometer. Se o banco vaziar, a senha real de todos os usuários fica exposta.

Errado:

email: ana@example.com
password: ana123

Correto:

email: ana@example.com
password_hash: \$2b\$12\$EixZaYVK1fsbw1ZfbX3OXe...

A API nunca precisa saber a senha original depois do cadastro. Ela só precisa conseguir verificar se uma senha digitada corresponde ao hash salvo.

4. Hash não $\emptyset$ criptografia reversível

Criptografia reversível transforma um dado em algo ilegível e depois permite voltar ao dado original usando uma chave.

Hash de senha não deve ser reversível.

No login, o processo não $\emptyset$:


```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

Payload

Contém claims, ou seja, declarações sobre o token.

```
{
  "sub": "123",
  "role": "admin",
  "type": "access",
  "exp": 1700000000
}
```

Signature

É a assinatura do token. Ela permite verificar se o token foi alterado. Se alguém muda o payload sem conhecer a chave secreta, a assinatura não bate.

8. JWT não assinado, não necessariamente criptografado

Este ponto é crítico: um JWT comum, JWS, não assinado, mas seu conteúdo pode ser lido por qualquer pessoa que tenha o token.

Portanto, não coloque no payload:

- senha;
- hash da senha;
- documento;
- cartão;
- segredo interno;
- dados pessoais desnecessários;
- permissões extremamente sensíveis sem checagem adicional no banco.

Coloque apenas o necessário para identificar o usuário e processar autorização básica.

9. Claims importantes

A RFC 7519 define algumas claims registradas. As mais importantes para APIs são:

sub

Subject. Representa o dono do token. Em APIs, normalmente é o id do usuário.

```
{"sub": "42"}
```

É comum usar string, mesmo que o id no banco seja inteiro.

exp

Expiration time. Define quando o token expira. Uma API deve rejeitar tokens expirados.

```
{"exp": 1700000000}
```

iat

Issued at. Momento em que o token foi emitido.

nbf

Not before. O token só é válido depois desse horário.

iss

Issuer. Quem emitiu o token. Útil quando há múltiplos emissores ou ambientes.

aud

Audience. Para quem o token foi emitido. Muito importante em arquiteturas com múltiplos serviços.

jti

JWT ID. Identificador único do token. Útil para revogação, auditoria e blacklist/allowlist.

10. Access token e refresh token

Um erro comum é usar um único token longo para tudo. Em projetos reais, é melhor separar responsabilidades.

Access token

- Usado para acessar rotas protegidas.
- Deve durar pouco.
- Pode carregar 'sub', 'role', 'type=access', 'exp'.
- Se for roubado, o dano é limitado pelo tempo curto de expiração.

Refresh token

- Usado para obter novo access token.
- Dura mais.
- Não deve ser usado para acessar rotas de negócio.
- Idealmente deve ser armazenado no banco de forma revogável.

Modelo comum:

access token: 15 minutos

refresh token: 7 dias

Quando o access token expira, o cliente chama '/auth/refresh' usando o refresh token.

11. Stateless vs stateful

JWT costuma ser descrito como stateless. Isso significa que o servidor consegue validar o token sem consultar uma tabela de sessões.

Isso tem vantagens:

- Menos consulta ao banco para validar token.
- Fácil escalar múltiplas instâncias da API.
- Bom para arquiteturas distribuídas.

Mas tem desvantagens:

- Revogar token antes do 'exp' não é trivial.
- Mudanças de permissão podem demorar a refletir se você confiar só no token.
- Logout real exige estratégia adicional.

Por isso, em projetos reais, é comum usar access token stateless e refresh token stateful. Ou seja: access token curto, refresh token salvo no banco.

12. Autorização por roles: RBAC

RBAC significa Role-Based Access Control. A ideia é simples: usuários recebem papéis, e papéis recebem permissões.

Exemplo básico:

admin -> pode criar, listar, editar e deletar
manager -> pode listar e editar
user -> pode listar seus próprios dados

Em um projeto pequeno, você pode checar diretamente a role:

```
require_role("admin")
```

Em um projeto maior, talvez seja melhor ter permissões granulares:

```
courses:create  
courses:read  
courses:update  
courses:delete  
users:manage
```

Roles são simples. Permissions são mais flexíveis. Para começar, roles funcionam bem.

13. Autorização deve acontecer no servidor

Nunca confie no frontend para autorização.

O frontend pode esconder um botão "Deletar usuário" de quem não é admin. Isso melhora UX, mas não a segurança. Qualquer pessoa pode chamar a API diretamente com curl, Postman ou script.

A regra real precisa estar na API.

14. Princípio do menor privilégio

Um usuário deve ter apenas as permissões necessárias para fazer seu trabalho.

Se uma rota só precisa de usuário logado, não exija admin. Se uma rota altera dados globais, não permita qualquer usuário logado. Se uma rota acessa dados próprios, verifique propriedade do recurso, não apenas role.

Exemplo:

```
GET /users/me -> usuário logado  
GET /users/{id} -> admin ou o próprio usuário  
DELETE /users/{id} -> admin
```

15. Status HTTP corretos

Use status codes de forma consistente:

400 Bad Request -> entrada inválida ou regra de negócio simples
401 Unauthorized -> não autenticado ou token inválido
403 Forbidden -> autenticado, mas sem permissão
404 Not Found -> recurso inexistente
409 Conflict -> conflito, como e-mail já cadastrado

401 significa "não sei quem você é" ou "sua credencial não é válida". 403 significa "sei quem você é, mas você não pode fazer isso".

16. Ameaças comuns

Token roubado

Se alguém rouba um bearer token válido, pode usá-lo até expirar. Mitigações:

- Access token curto.
- HTTPS obrigatório.
- Não salvar token em locais inseguros.
- Refresh token revogável.

Algoritmo incorreto

A API deve aceitar apenas algoritmos esperados. Nunca decodifique token aceitando qualquer algoritmo vindo do header sem validação.

Secret fraca

HS256 depende de uma chave secreta forte. Use segredo longo e aleatório.

Token sem expiração

Token sem 'exp' pode durar para sempre. Em APIs reais, sempre use expiração.

Role obsoleta no token

Se a role muda no banco, um token antigo ainda pode carregar role antiga. Mitigações:

- Access token curto.
- Buscar usuário atual no banco.
- Usar token version.
- Revogar tokens em mudanças críticas.

IDOR

IDOR significa Insecure Direct Object Reference. Exemplo: usuário autenticado acessa '/orders/123' mesmo não sendo dono do pedido.

Roles não resolvem isso sozinhas. Você precisa verificar propriedade do recurso.

Parte 2: arquitetura prática em FastAPI

17. Estrutura de arquivos recomendada

Uma estrutura madura, ainda simples:

```
app/  
  main.py  
  core/  
    config.py  
    database.py  
    security.py  
    auth.py  
  models/  
    user_model.py  
    refresh_token_model.py  
  schemas/  
    auth_schema.py  
    user_schema.py  
  repositories/  
    user_repository.py
```

```

    refresh_token_repository.py
services/
    auth_service.py
    user_service.py
routers/
    auth_routes.py
    user_routes.py
    course_routes.py

```

Separação:

- 'security.py': funções puras de segurança.
- 'auth.py': dependências do FastAPI.
- 'auth_service.py': regras de login, refresh, logout.
- 'auth_routes.py': endpoints HTTP.
- 'repositories': acesso ao banco.
- 'schemas': entrada e saída.

Essa divisão evita que uma rota vire um arquivo enorme misturando HTTP, banco, senha e JWT.

18. Configuração

Arquivo '.env':

```

SECRET_KEY=gere_uma_chave_grande_e_aleatoria
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=15
REFRESH_TOKEN_EXPIRE_DAYS=7
JWT_ISSUER=fastapi-api
JWT_AUDIENCE=fastapi-clients

```

'app/core/config.py':

```

from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    secret_key: str
    algorithm: str = "HS256"
    access_token_expire_minutes: int = 15
    refresh_token_expire_days: int = 7
    jwt_issuer: str = "fastapi-api"
    jwt_audience: str = "fastapi-clients"

    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8")

settings = Settings()

```

Para projeto real, 'issuer' e 'audience' ajudam a evitar que tokens emitidos para um contexto sejam aceitos em outro.

19. Modelo de usuário

```

from sqlalchemy import Boolean, Column, DateTime, Integer, String, func
from app.core.database import Base

class UserModel(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, autoincrement=True)
    name = Column(String, nullable=False)

```



```

email = Column(String, unique=True, index=True, nullable=False)
password_hash = Column(String, nullable=False)
role = Column(String, nullable=False, default="user", server_default="user")
active = Column(Boolean, nullable=False, default=True, server_default="1")
created_at = Column(DateTime, server_default=func.now(), nullable=False)
updated_at = Column(DateTime, onupdate=func.now())

```

Prefira o nome 'password_hash' no model. Ele comunica melhor que o campo não guarda senha pura.

Roles possíveis:

```

admin
manager
user

```

Para sistemas maiores, roles podem virar tabela separada.

20. Schemas de usuário

```

from typing import Literal
from pydantic import BaseModel, EmailStr, Field

Role = Literal["admin", "manager", "user"]

class UserCreate(BaseModel):
    name: str = Field(min_length=2, max_length=120)
    email: EmailStr
    password: str = Field(min_length=8, max_length=128)

class UserUpdate(BaseModel):
    name: str | None = Field(default=None, min_length=2, max_length=120)
    password: str | None = Field(default=None, min_length=8, max_length=128)

class UserAdminUpdate(BaseModel):
    name: str | None = None
    role: Role | None = None
    active: bool | None = None

class UserResponse(BaseModel):
    id: int
    name: str
    email: EmailStr
    role: str
    active: bool

class Config:
    from_attributes = True

```

Separe update comum de update administrativo. Um usuário comum não deve conseguir alterar a própria role enviando JSON manualmente.

21. Segurança: bcrypt e JWT

'app/core/security.py':

```

from datetime import datetime, timedelta, timezone
from typing import Any
from uuid import uuid4

import bcrypt

```

```

from jose import JWTError, jwt

from app.core.config import settings

def hash_password(password: str) -> str:
    salt = bcrypt.gensalt()
    hashed = bcrypt.hashpw(password.encode("utf-8"), salt)
    return hashed.decode("utf-8")

def verify_password(plain_password: str, password_hash: str) -> bool:
    return bcrypt.checkpw(
        plain_password.encode("utf-8"),
        password_hash.encode("utf-8"),
    )

def create_jwt_token(subject: str, token_type: str, expires_delta: timedelta, extra_claims:
dict[str, Any] | None = None) -> str:
    now = datetime.now(timezone.utc)
    payload = {
        "sub": subject,
        "type": token_type,
        "iat": now,
        "nbf": now,
        "exp": now + expires_delta,
        "iss": settings.jwt_issuer,
        "aud": settings.jwt_audience,
        "jti": str(uuid4()),
    }

    if extra_claims:
        payload.update(extra_claims)

    return jwt.encode(payload, settings.secret_key, algorithm=settings.algorithm)

def create_access_token(user_id: int, role: str) -> str:
    return create_jwt_token(
        subject=str(user_id),
        token_type="access",
        expires_delta=timedelta(minutes=settings.access_token_expire_minutes),
        extra_claims={"role": role},
    )

def create_refresh_token(user_id: int) -> str:
    return create_jwt_token(
        subject=str(user_id),
        token_type="refresh",
        expires_delta=timedelta(days=settings.refresh_token_expire_days),
    )

def decode_token(token: str) -> dict[str, Any] | None:
    try:
        return jwt.decode(
            token,
            settings.secret_key,
            algorithms=[settings.algorithm],
            audience=settings.jwt_audience,
            issuer=settings.jwt_issuer,
        )
    except JWTError:
        return None

```

Aqui entramos em um padrão mais realista:

- 'iat': quando foi emitido.
- 'nbf': antes de quando no vale.
- 'iss': emissor esperado.
- 'aud': audi ncia esperada.
- 'jti': identificador  nico.
- 'type': separa access e refresh.

22. Repository de usu rio

```
from sqlalchemy import select
from sqlalchemy.orm import Session

from app.models.user_model import UserModel

class UserRepository:
    def find_by_id(self, db: Session, user_id: int) -> UserModel | None:
        return db.scalar(select(UserModel).where(UserModel.id == user_id))

    def find_by_email(self, db: Session, email: str) -> UserModel | None:
        return db.scalar(select(UserModel).where(UserModel.email == email))

    def create(self, db: Session, user: UserModel) -> UserModel:
        db.add(user)
        db.commit()
        db.refresh(user)
        return user

    def save(self, db: Session, user: UserModel) -> UserModel:
        db.add(user)
        db.commit()
        db.refresh(user)
        return user
```

O repository no sabe o que   login. Ele apenas busca e persiste.

23. Auth schemas

```
from pydantic import BaseModel, EmailStr

class LoginRequest(BaseModel):
    email: EmailStr
    password: str

class RefreshRequest(BaseModel):
    refresh_token: str

class TokenPairResponse(BaseModel):
    access_token: str
    refresh_token: str
    token_type: str = "bearer"

class AccessTokenResponse(BaseModel):
    access_token: str
    token_type: str = "bearer"
```

Voc  pode devolver novo refresh token a cada refresh ou apenas novo access token. Rotacionar refresh token   mais seguro, mas exige persist ncia e revoga o.

24. Auth service

'app/services/auth_service.py':

```

from fastapi import HTTPException, status
from sqlalchemy.orm import Session

from app.core.security import create_access_token, create_refresh_token, decode_token,
verify_password
from app.repositories.user_repository import UserRepository

class AuthService:
    def __init__(self):
        self.users = UserRepository()

    def login(self, db: Session, email: str, password: str):
        user = self.users.find_by_email(db, email)

        if not user or not verify_password(password, user.password_hash):
            raise HTTPException(
                status_code=status.HTTP_401_UNAUTHORIZED,
                detail="Invalid credentials",
                headers={"WWW-Authenticate": "Bearer"},
            )

        if not user.active:
            raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="Inactive user")

        return {
            "access_token": create_access_token(user.id, user.role),
            "refresh_token": create_refresh_token(user.id),
            "token_type": "bearer",
        }

    def refresh(self, db: Session, refresh_token: str):
        payload = decode_token(refresh_token)
        if not payload or payload.get("type") != "refresh":
            raise HTTPException(status_code=401, detail="Invalid refresh token")

        user = self.users.find_by_id(db, int(payload["sub"]))
        if not user or not user.active:
            raise HTTPException(status_code=401, detail="User not found or inactive")

        return {
            "access_token": create_access_token(user.id, user.role),
            "refresh_token": create_refresh_token(user.id),
            "token_type": "bearer",
        }

```

Em produção, o método 'refresh' deveria verificar se o refresh token existe no banco, se não foi revogado e se pertence ao usuário.

25. Dependências de autenticação

'app/core/auth.py':

```

from collections.abc import Callable

from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer
from sqlalchemy.orm import Session

from app.core.database import get_db
from app.core.security import decode_token
from app.models.user_model import UserModel
from app.repositories.user_repository import UserRepository

```

```

bearer_scheme = HTTPBearer()

def get_current_user(
    credentials: HTTPAuthorizationCredentials = Depends(bearer_scheme),
    db: Session = Depends(get_db),
) -> UserModel:
    payload = decode_token(credentials.credentials)

    if not payload or payload.get("type") != "access":
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid or expired access token",
            headers={"WWW-Authenticate": "Bearer"},
        )

    user_id = payload.get("sub")
    if not user_id:
        raise HTTPException(status_code=401, detail="Invalid token subject")

    user = UserRepository().find_by_id(db, int(user_id))
    if not user or not user.active:
        raise HTTPException(status_code=401, detail="User not found or inactive")

    return user

def require_roles(*allowed_roles: str) -> Callable:
    def checker(current_user: UserModel = Depends(get_current_user)) -> UserModel:
        if current_user.role not in allowed_roles:
            raise HTTPException(status_code=403, detail="Insufficient permissions")
        return current_user

    return checker

```

Essa é a ponte entre segurança e rotas. FastAPI executa a dependência antes da função da rota.

26. Rotas de autenticação

'app/routers/auth_routes.py':

```

from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session

from app.core.auth import get_current_user
from app.core.database import get_db
from app.models.user_model import UserModel
from app.schemas.auth_schema import LoginRequest, RefreshRequest, TokenPairResponse
from app.schemas.user_schema import UserResponse
from app.services.auth_service import AuthService

router = APIRouter(prefix="/auth", tags=["Auth"])

@router.post("/login", response_model=TokenPairResponse)
def login(payload: LoginRequest, db: Session = Depends(get_db)):
    return AuthService().login(db, payload.email, payload.password)

@router.post("/refresh", response_model=TokenPairResponse)
def refresh(payload: RefreshRequest, db: Session = Depends(get_db)):
    return AuthService().refresh(db, payload.refresh_token)

@router.get("/me", response_model=UserResponse)
def me(current_user: UserModel = Depends(get_current_user)):
    return current_user

```

27. Rotas protegidas por autenticação

Uma rota que exige apenas login:

```
@router.get("/profile", response_model=UserResponse)
def profile(current_user: UserModel = Depends(get_current_user)):
    return current_user
```

Uma rota de leitura protegida:

```
@router.get("/courses")
def list_courses(current_user: UserModel = Depends(get_current_user)):
    return CourseService().list_all()
```

28. Rotas protegidas por role

Uma rota apenas para admin:

```
@router.delete("/users/{user_id}")
def delete_user(
    user_id: int,
    current_user: UserModel = Depends(require_roles("admin")),
):
    return UserService().delete(user_id)
```

Uma rota para admin ou manager:

```
@router.post("/courses")
def create_course(
    payload: CourseCreate,
    current_user: UserModel = Depends(require_roles("admin", "manager")),
):
    return CourseService().create(payload)
```

29. Autorização por propriedade do recurso

Roles não resolvem tudo. Muitas regras dependem de propriedade.

Exemplo:

Usuário comum pode ver o próprio perfil.

Admin pode ver qualquer perfil.

Código:

```
@router.get("/users/{user_id}", response_model=UserResponse)
def get_user(
    user_id: int,
    current_user: UserModel = Depends(get_current_user),
):
    if current_user.role != "admin" and current_user.id != user_id:
        raise HTTPException(status_code=403, detail="Insufficient permissions")

    return UserService().get_by_id(user_id)
```

Essa proteção evita IDOR.

30. Registrando routers

'app/main.py':

```
from fastapi import FastAPI
from app.routers import auth_routes, user_routes, course_routes

app = FastAPI(title="Real FastAPI Auth API")

app.include_router(auth_routes.router)
app.include_router(user_routes.router)
app.include_router(course_routes.router)

@app.get("/health")
def health():
    return {"status": "ok"}
```

31. Fluxo de teste manual

1. Criar usuário.
2. Promover usuário para admin, se necessário.
3. Fazer login.
4. Copiar access token.
5. Chamar '/auth/me' com Bearer token.
6. Chamar rota comum protegida.
7. Chamar rota admin.
8. Testar com usuário sem permissão.
9. Esperar access token expirar.
10. Chamar '/auth/refresh'.

cURL login:

```
curl -X POST http://127.0.0.1:8000/auth/login \
-H "Content-Type: application/json" \
-d '{"email":"admin@example.com","password":"admin12345"}'
```

cURL protegido:

```
curl http://127.0.0.1:8000/auth/me \
-H "Authorization: Bearer ACCESS_TOKEN_AQUI"
```

32. Swagger e Authorize

FastAPI gera documentação em:

/docs

Se você usa 'HTTPBearer', o Swagger mostra um esquema Bearer. Cole o token conforme a interface pedir. Em muitos casos, basta colar o token sem 'Bearer'; em outros, cole 'Bearer <token>'. Verifique como seu esquema aparece.

Se quiser integração OAuth2 mais clássica, use 'OAuth2PasswordBearer'. Ele é muito usado nos exemplos oficiais do FastAPI.

33. Refresh token robusto

A versão simples gera refresh token JWT e valida assinatura/expiração. Para projeto real, o

ideal Ø persistir refresh tokens.

Modelo:

```
class RefreshTokenModel(Base):
    __tablename__ = "refresh_tokens"

    id = Column(Integer, primary_key=True)
    user_id = Column(ForeignKey("users.id"), nullable=False)
    token_hash = Column(String, nullable=False)
    jti = Column(String, unique=True, nullable=False)
    revoked = Column(Boolean, default=False)
    expires_at = Column(DateTime, nullable=False)
    created_at = Column(DateTime, server_default=func.now())
```

Boas prÆticas:

- Nªo salvar refresh token puro no banco; salve hash.
- Revogar token no logout.
- Rotacionar refresh token a cada uso.
- Detectar reutilizaªo de refresh token antigo.

34. Logout com JWT

Com access token stateless, logout nªo invalida automaticamente um token jÆ emitido.

Possibilidades:

1. Logout remove/revoga refresh token.
2. Access token continua vÆlido atØ expirar.
3. Access token curto reduz o risco.
4. Para invalidaªo imediata, use blacklist por 'jti' ou token version no usuÆrio.

Para a maioria das APIs, access token curto + refresh token revogÆvel Ø um bom equilÍbrio.

35. Token version

Uma tØcnica cØtil Ø armazenar 'token_version' no usuÆrio.

```
users.token_version = 3
JWT inclui token_version = 3
```

Se o usuÆrio troca senha ou perde acesso, incremente 'token_version'. Tokens antigos deixam de bater com o valor atual.

Isso exige consultar o banco ao validar token, mas muitas APIs jÆ fazem isso para checar usuÆrio ativo.

36. Scopes vs roles

FastAPI tem suporte a OAuth2 scopes. Scopes representam permissies especÍficas.

Exemplos:

```
users:read
users:write
courses:create
courses:delete
```

Roles agrupam permissies:

admin -> users:read, users:write, courses:create, courses:delete
manager -> courses:create
user -> courses:read

Para sistemas pequenos, roles são suficientes. Para sistemas grandes, scopes/permissions dão mais controle.

37. Onde guardar tokens no cliente

Depende do tipo de cliente.

SPA no navegador

Opções comuns:

- Memória: mais seguro contra persistência, perde ao recarregar.
- HttpOnly cookie: protege contra leitura por JavaScript, exige cuidado com CSRF.
- localStorage: simples, mas mais exposto em XSS.

Não existe opção perfeita. O desenho depende do risco e da arquitetura.

Aplicativo mobile

Use armazenamento seguro do sistema, como Keychain/Keystore.

Backend chamando backend

Use segredo de serviço, OAuth2 client credentials, mTLS ou estratégia própria de serviço.

38. CORS, cookies e CSRF

Se você usa Bearer token em header Authorization, CSRF tende a ser menos central, porque navegador não adiciona esse header automaticamente.

Se você usa cookies para autenticação, precisa pensar em CSRF:

- SameSite.
- CSRF token.
- Verificação de origem.

CORS não é autenticação nem autorização. CORS controla quais origens o navegador pode chamar. Ele não impede chamadas diretas por servidor, curl ou Postman.

39. Testes automatizados essenciais

Teste pelo menos:

- Cadastro salva senha hashada.
- Login com senha correta retorna tokens.
- Login com senha errada retorna 401.
- Token expirado retorna 401.
- Refresh token gera novo access token.
- Rota protegida sem token falha.
- Rota protegida com token válido passa.
- Rota admin com user comum retorna 403.
- Usuário inativo não acessa.
- Usuário não consegue acessar recurso de outro usuário.

40. Checklist de produção

Antes de levar para produção:

- SECRET_KEY forte e fora do Git.
- HTTPS obrigatório.

- Access token curto.
- Refresh token revogável.
- Hash de senha com algoritmo apropriado.
- Rate limit em login.
- Mensagens de login genéricas.
- Logs de autenticação sem vazarem senha/token.
- Autorização no backend, não sóno frontend.
- Proteção contra IDOR.
- Testes automatizados.
- Plano de rotação de segredo.
- Política para primeiro admin.
- Migrações de banco versionadas.

41. Erros comuns

Colocar senha no JWT

Nunca faça isso.

Salvar senha em texto puro

Sempre gere hash antes de salvar.

Aceitar refresh token em rota comum

Use claim 'type' e rejeite qualquer token que não seja 'access'.

Confiar apenas na role do token

Para sistemas sensíveis, consulte o usuário no banco e confirme se ainda está ativo.

Não verificar propriedade do recurso

Usuário logado não deve automaticamente acessar qualquer id.

Token sem expiração

Sempre inclua 'exp'.

Secret fraca

Use segredo longo e aleatório.

42. Implementação mínima completa

A implementação mínima realista tem:

- 'hash_password'.
- 'verify_password'.
- 'create_access_token'.
- 'create_refresh_token'.
- 'decode_token'.
- '/auth/login'.
- '/auth/refresh'.
- '/auth/me'.
- 'get_current_user'.
- 'require_roles'.
- schemas sem expor senha.
- roles no usuário.
- proteção de rotas.

43. Modelo mental final

Autenticação ≠ identidade. Autorização ≠ permissão. Senha não se guarda, se hasheia. JWT não é segredo criptografado, é declaração assinada. Access token abre portas por pouco

tempo. Refresh token renova acesso e precisa de controle maior. Role é uma regra ampla de autorização. Permissão é uma regra granular. FastAPI 'Depends' é a forma elegante de aplicar segurança nas rotas. Backend sempre decide se a ação é permitida.

Quando esse modelo mental fica claro, o código deixa de parecer mágico. Cada função passa a ter um papel específico na cadeia de segurança.